VIT-AP UNIVERSITY

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **2**
Topic : **1**

NoSQL Data model: Aggregate Models

Objectives

This session will give the knowledge about

- Aggregate Models
- Types of NoSQL
- Key-value stores

Aggregate Models

- A data architecture pattern is a consistent way of representing data in a regular structure that will be stored in memory.
- An aggregate is a collection of data that we interact with as a unit.
- These units of data or aggregates form the boundaries for ACID operations with the database, Key-value, Document, Column-family and Graph-Based databases are grouped as aggregate-oriented database.

Aggregate Models

Aggregates make it easier for the database to manage data storage over clusters.

Aggregate-oriented databases work best when most data interaction is done with the same aggregate,

Example: when there is need to get an order and all its details, it better to store order as an aggregate object.

Aggregate-oriented databases make inter-aggregate relationships more difficult to handle than intra-aggregate relationships.

Distribution Models

Aggregate oriented databases make distribution of data easier. There are two styles of distributing data:

- Sharding: Sharding distributes different data across multiple servers, so each server acts as the single source for a subset of data.
- Replication: Replication copies data across multiple servers, so each bit of data can be found in multiple places. Replication comes in two forms,
  - Master-slave replication makes one node the authoritative copy that handles writes while slaves synchronize with the master and may handle reads.
  - Peer-to-peer replication allows writes to any node; the nodes coordinate to synchronize their copies of the data.

NoSQL data architecture patterns

One of the challenges for users of NoSQL systems is there are many different architectural patterns from which to choose. There are 4 basic types of NoSQL databases:

- Key-Value Store – It has a Big Hash Table of keys & values {**Example- Riak, Redis Amazon S3 (Dynamo)**}
- Document-based Store - It stores documents made up of tagged elements. {**Example- Mongo DB, CouchDB**}
- Column-based Store **-** Each storage block contains data from only one column, {**Example- HBase, Cassandra**}
- Graph-based - A network database that uses edges and nodes to represent and store data. {**Example- Neo4J**}

NoSQL data architecture patterns

Pattern name	Description	Typical uses
Key-value store	A simple way to associate a large data file with a simple text string	Dictionary, image store, document/file store, query cache, lookup tables
Graph store	A way to store nodes and arcs of a graph	Social network queries, friend-of-friends queries, inference, rules system, and pattern matching
Column family (Bigtable) store	A way to store sparse matrix data using a row and a column as the key	Web crawling, large sparsely populated tables, highly-adaptable systems, systems that have high variance
Document store	A way to store tree-structured hierarchical information in a single unit	Any data that has a natural container structure including office documents, sales orders, invoices, product descriptions, forms, and web pages; popular in publishing, document exchange, and document search

NoSQL data architecture patterns

Key-Value	Document Oriented
Caching: Used for caching data, **Session Management**: Storing session information in web applications. **Log Systems**: Managing user logs in e-commerce platforms.	**Content Management Systems (CMS)**: Managing and storing varied content like articles, blogs, etc., **User Profiles**: Storing user data with varying attributes. **Real-time Analytics**: Used for applications requiring real-time data processing and storage.
Columnar model	**Graph Base**
Data Warehousing: large-scale data analysis and reporting are required. **Time-Series Data**: Efficiently stores and queries time-series data, often used in IoT or financial data applications. **OLAP (Online Analytical Processing)**: Optimized for analytical queries involving large datasets.	**Social Networks**: Modeling and querying social connections, e.g., Facebook's Graph **Recommendation Engines**: recommending products, friends, or content based on relationships **Fraud Detection**: Identifying patterns and connections that might indicate fraudulent behavior. **Knowledge Graphs**

Key-value stores

A key-value store is a simple database that when presented with a simple string (the key) returns an arbitrary large BLOB of data (the value).

Key-value stores have no query language; they provide a way to add and remove key-value pairs into/from a database.

A key-value store is like a dictionary.

The dictionary is a simple key-value store where word entries represent keys and definitions represent values.

Key-value stores

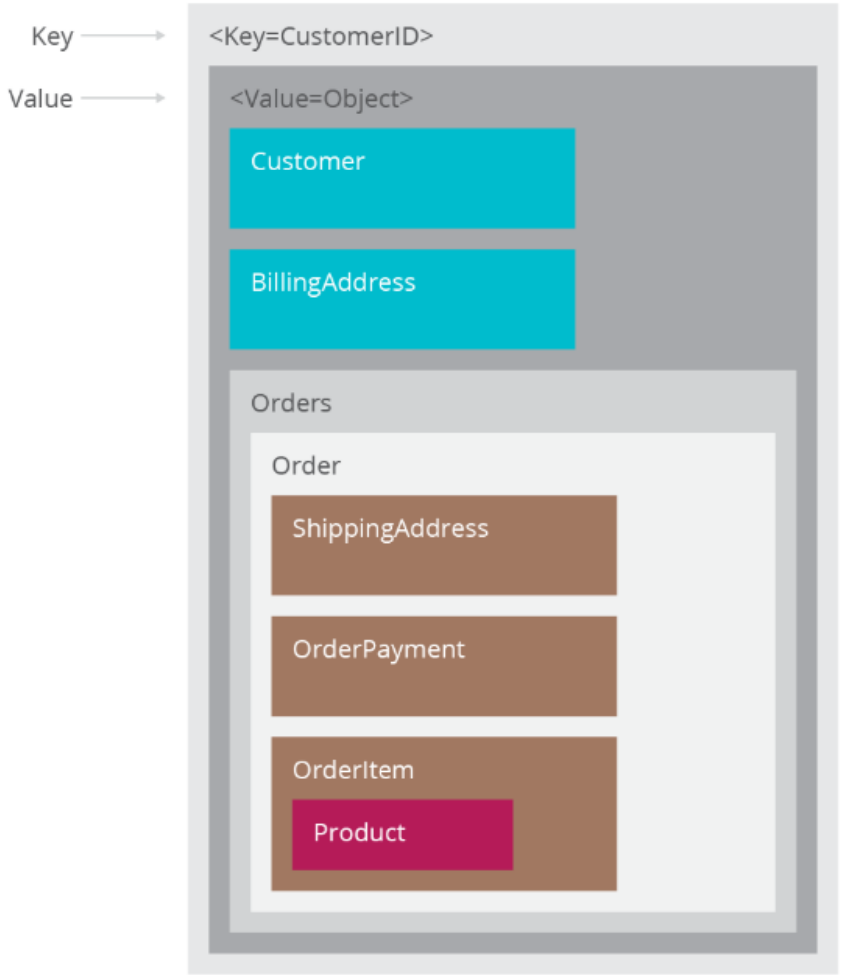
The key value type basically, uses a hash table in which there exists a unique key and a pointer to a particular item of data.

A bucket is a logical group of keys – but they don't physically group the data. There can be identical keys in different buckets.

The key can be synthetic or auto-generated while the value can be String, JSON, BLOB (basic large object) etc.

To read a value you need to know both the key and the bucket because the real key is a hash (Bucket + Key).

Key-value stores – Data types



Key-value stores – Data types

The key in a key-value store is flexible and can be represented by many formats:

- Logical path names to images or files
- Artificially generated strings created from a hash of the value
- REST web service calls
- SQL queries

Values, like keys, are also flexible and can be any BLOB of data, such as images, web pages, documents, or videos.

Sample items in a key-value store

	Key	Value
Image name →	image-12345.jpg	Binary image file
Web page URL →	http://www.example.com/my-web-page.html	HTML of a web page
File path name →	N:/folder/subfolder/myfile.pdf	PDF document
MD5 hash →	9e107d9d372bb6826bd81d3542a419d6	The quick brown fox jumps over the lazy dog
REST web service call →	view-person?person-id=12345&format=xml	<Person><id>12345</id .</Person>
SQL query →	SELECT PERSON FROM PEOPLE WHERE PID="12345"	<Person><id>12345</id .</Person>

Key-value stores - Example

Consider the data subset represented in the following table. Here the key is the state name, while the value is a list of addresses of VIT campuses.

KEY	VALUE
Tamilnadu	{“Gorbachev Rd, Vellore, Tamil Nadu 632014”}
Andhra Pradesh	{“VIT-AP University, G-30, Inavolu, Beside AP Secretariat Amaravati, Andhra Pradesh 522237”}
Bopal	{“Kotri Kalan, Ashta, Near, Indore Road, Bhopal, Madhya Pradesh 466114”}

Key-value stores - Operations

The best way to think about using a key-value store is to visualize a single table with two columns.

The first column is called the key and the second column is called the value.

There are three operations performed on a key-value store:

- put,
- get, and
- delete.

Key-value stores - Operations

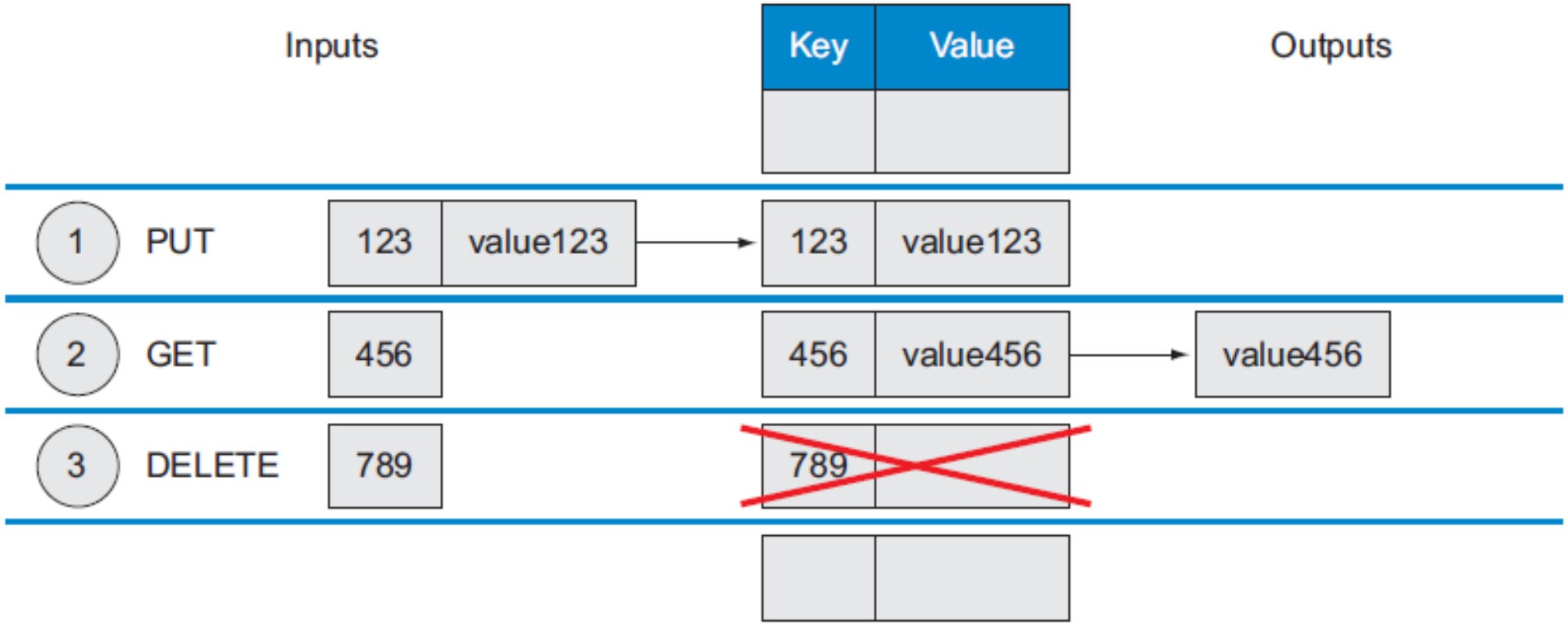
Instead of using a query language, application developers access and manipulate a key-value store with the put, get, and delete functions, as shown here:

1. put($key as xs:string, $value as item()) adds a new key-value pair to the table and will update a value if this key is already present.

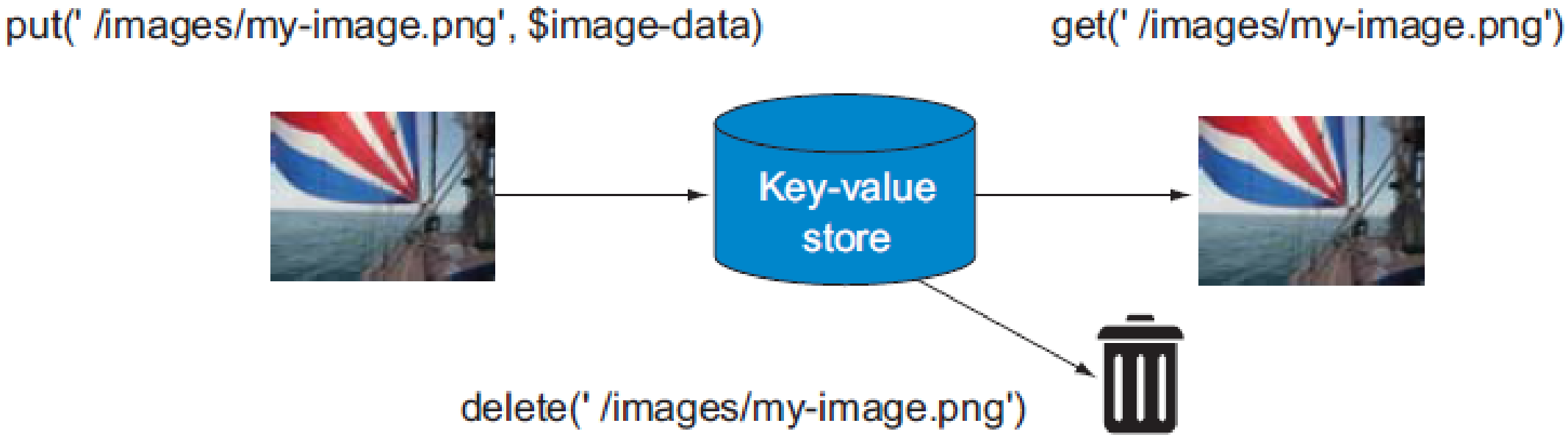
2. get($key as xs:string) as item() returns the value for any given key, or it may return an error message if there's no key in the key-value store.

3. delete($key as xs:string) removes a key and its value from the table, or it many return an error message if there's no key in the key-value store.

Key-value stores - Operations



Key-value stores – Operations - Output



Key-value stores - Operations

In addition to the put, get, and delete API, a key-value store has two rules: distinct keys and no queries on values:

1. Distinct keys - You can never have two rows with the same key-value. This means that all the keys in any given key-value store are unique.

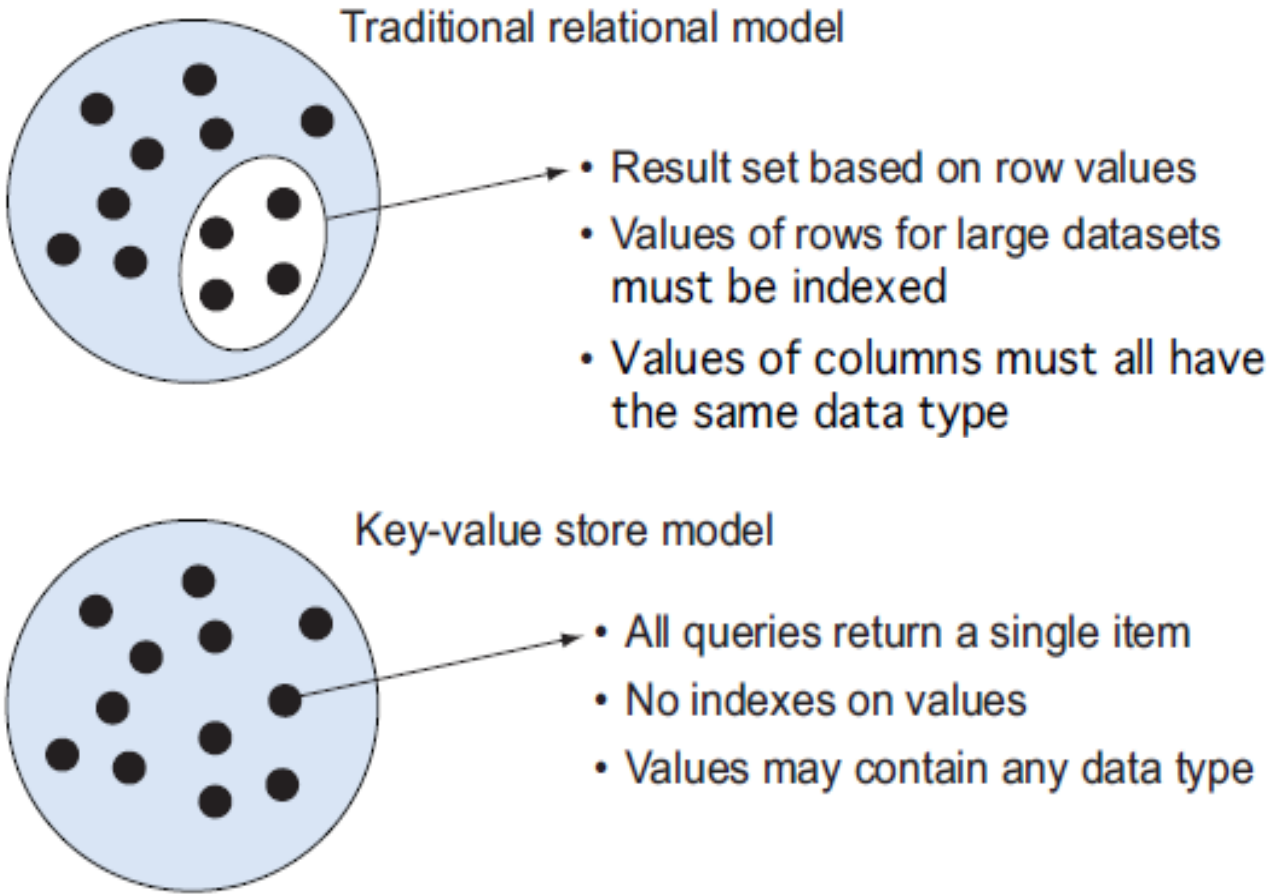
2. No queries on values - You can't perform queries on the values of the table.

Key-value stores - Operations

A key-value store prohibits where clause type of operation, as you can't select a key-value pair using the value.

The key-value store resolves the issues of indexing and retrieval in large datasets by transferring the association of the key with the value to the application layer, allowing the key-value store to retain a simple and flexible structure.

Key-value stores - Operations



Benefits of using a key-value store

NoSQL have simplicity and generality to save time and money by moving the focus from architectural design to reducing the data services costs through

- Precision service levels
- Precision service monitoring and notification
- Scalability and reliability
- Portability and lower operational costs

Benefits - Precision service levels

For any data service the precision service levels might specify

- The maximum read time to return a value and write time to store a new key-value pair
- How many reads per second the service must support and writes per second the service must support
- How many duplicate copies of the data should be created for enhanced reliability
- Whether the data should be duplicated across multiple geographic regions if some data centers experience failures
- Whether to use transaction guarantees for consistency of data or whether eventual consistency is adequate

Benefits - Precision service levels

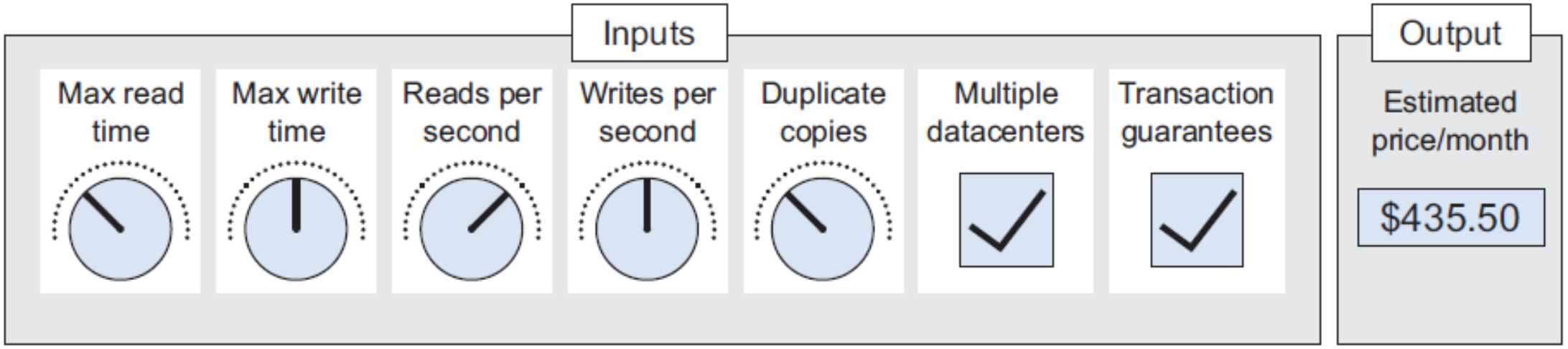
You can configure your system to use a simple input form for setting up and allocating resources to new data services.

By changing the information using the form, you can quickly change the number of resources allocated to the service.

NoSQL data services can be adjusted like the tuning knobs on a radio.

Each knob can individually be adjusted to control how many resources are used to provide service guarantees. The more resources you use, the higher the cost will be.

Benefits - Precision service levels



Benefits - Precision service monitoring and notification

In addition to specifying service levels, you can also invest in tools to monitor your service level.

When you configure the number of reads per second a service performs, setting the parameter too low may lead a delay during peak times.

By using a simple API, detailed reports showing the expected versus actual loads can point you to system bottlenecks that may need additional resource adjustments.

Example: Automatic notification systems can also trigger email messages when the volume of reads or writes exceeds a threshold within a specified period of time.

Benefits - Scalability and reliability

When a database interface is simple, the resulting systems can have higher scalability and reliability.

This means you can tune any solution to the desired requirements.

Keeping an interface simple allows novice as well as advanced data modelers to build systems that utilize this power.

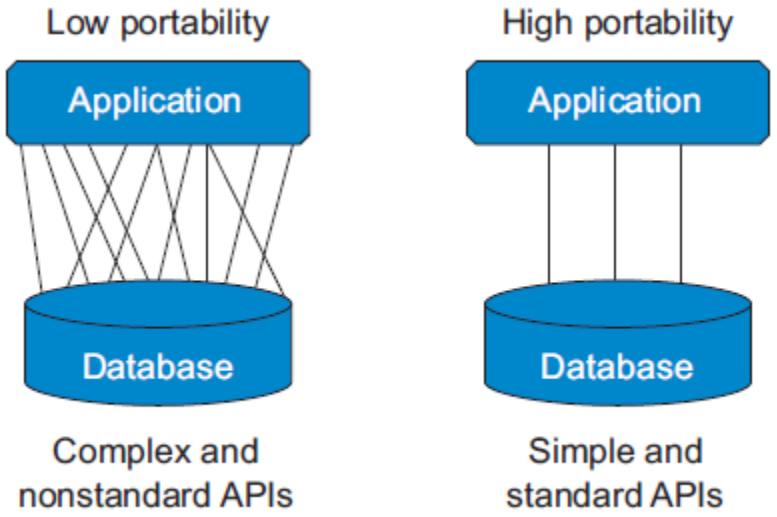
Benefits - Scalability and reliability

A simple interface allows you to focus on load and stress testing and monitoring of service levels.

Because a key-value store is simple to set up, you can spend more time looking at how long it takes to put or get 10,000 items.

It also allows you to share these load- and stress-testing tools with other members of your development team.

Benefits - Portability and lower operational costs



Portability of any application depends on database interface complexity.

The low-portability system on the left has many complex interfaces between the application and the database, and porting the application between two databases might be a complex process requiring a large testing effort.

In contrast, the high portability application on the right only uses a few standardized interfaces, such as put, get, and delete, and could be quickly ported to a new database with lower testing cost.

Case Studies

The first, storing web pages in a key-value store, shows how web search engines such as Google easily store entire websites in a key-value store. So if you want to store external websites in your own local database, this type of key-value store is for you.

The second use case, Amazon simple storage service (S3), shows how you can use a key-value store like S3 as a repository for your content in the cloud. If you have digital media assets such as images, music, or video, you should consider using a key-value store to increase the reliability and performance for a fraction of the cost.

Summary

This session will give the knowledge about

- Aggregate Models
- Types of NoSQL
- Key-value stores